

*A connected world
Integrated modeling
Needs specialized tools*

From time to time, great debates arise in discussion forums about the pros and cons of various data modeling tools; inevitably, one or more people say that their favorite tools are the whiteboard and sticky notes. The whiteboard and sticky notes are great tools for workshop use, enabling rapid development of a model by a group of people without technical data modeling skills. Once a data model has been fleshed out on a whiteboard or a wall, it usually needs to be recorded in a more permanent fashion. Very often, the first choice is the old favorite analysis tool, the spreadsheet. Like the whiteboard and sticky notes, the spreadsheet is a great way of capturing information during and after a workshop, and doesn't require expensive software licenses or special skills. By itself, however, it only provides a fraction of the capabilities required to construct, validate, communicate and use a data model effectively.

Why not use a drawing tool for data modeling?

You may be tempted to use a drawing tool, such as Microsoft® PowerPoint® or Visio®, or Smart Draw®. These products allow you to draw lines and boxes, and add text to diagrams; they can create diagrams that communicate ideas visually, and maybe even capture some additional information about some of the symbols. However, each diagram stands alone: if an entity appears on five diagrams, there is no connection between those five entity symbols. There is no way to ensure that those five diagrams are consistent in how they depict that entity, nor is there a way of knowing whether or not the entity also appears on a sixth diagram that you haven't been told about. There is also no link from that entity to any database tables that were generated from it, and no link to the business processes that manage that entity. In short, the entities appear to live in a disconnected world. When it comes to managing change, that disconnected world gets very uncomfortable. As the models change, you have to manually change all the relevant diagrams and documents. Knowing which documents need to be changed and ensuring they remain consistent is a daunting task.

In Section III, you will see the different levels and types of data models, each with their own purposes and usage. There are dependencies between these different types of data models, between data models and other artifacts or models that represent other aspects of business and requirements, the enterprise and solutions architecture, and application design. The activities required to produce and manage data models are only part of a wider set of business and technology activities; integration with associated activities is key to the success of data modeling.

82 Data Modeling Made Simple with PowerDesigner

Without a tool that provides specialized support for data modeling, the data modeler cannot hope to work effectively in this environment.

Organizations take a variety of approaches to the enablement of data modeling, depending on the money available, and the perceived role of data models in their organization.

Key Features Needed in a Data Modeling Tool

With data modeling tools, you generally get what you pay for. The more you pay, the more features are provided by the product. The wider your use of data models within the organization, the more features you tend to need. If you're a data modeling novice, you should really read Sections I and II before reading this section.

In this section, we list some of the key features that we should expect a data modeling tool to provide. There is no hidden meaning in the order of the lists, other than to ensure that related features are close to each other. The features are grouped into five categories – Core Modeling, Usability, Interfaces and Integration, Tool Management and Communication, and Collaboration. Chapter 9 uses the same categories to analyze the features provided by PowerDesigner.

Core Modeling

- Construct different levels and types of data models, including any distinct notation and content required for each level
 - See section IV for details of the six types of data models
- Generate a new data model from selected content in an existing data model
 - create and maintain links between the original model and the generated model
 - includes transforming a logical data model into a physical data model, and vice versa
- Provide support for multiple data modeling notations, such as IDEF1X and Information Engineering
 - see Chapter 9 for more information about data modeling notation
- Create subsets of a data model using selected entities or tables
- Define validation and design rules, and use these to validate models
- Automatically correct validation errors, where appropriate
- Automatically enforce naming standards as model objects are created or modified
- View and edit multiple model objects in a single operation, similar to editing within a spreadsheet
- For physical data models, provide direct support for model objects specific to a technology, such as a given RDBMS or XML Schema Language
- Configurable support for technology objects (e.g. Creating generation capabilities for unsupported DBMS)
- Automated support for denormalization of physical data models, including rollback capabilities

- Record and manage the dependencies between objects in data models
 - for example, “which diagrams does an entity appear in?”, “which attributes are constrained by a given domain?”
 - include the ability to traverse dependencies between model objects and filter the information provided; for example, starting with a given domain, extract details of the attributes constrained by the domain, any business rules referenced by those attributes, and also information about the owners of the business rules
- Support for multiple versions of models and model objects
- Automation of common modeling tasks, such as merging or splitting an entity, or converting a many-to-many relationship into an entity and two relationships
- Facilities for the re-use of modeling patterns and reference models within data models
- Record the vocabulary of the business as terms, and enforce naming standards based on those terms
- Replicate attributes between entities (ditto columns and tables)
- Replicate objects between models
- Create and manage shortcuts to objects in other models

Usability

- Control over the windows and toolbars displayed, and their position and size
- Auto-layout of selected parts of a diagram or a complete diagram
- Control of the placement of symbols on a diagram by the analyst
- Control over the style and content of symbols by the analyst
- A single editable view of any object
- Flexible editing capabilities
 - allow the user to choose the properties to view and edit for a given type of object
 - provide multiple ways of editing object properties, allowing the user to choose the method most appropriate for them
 - include the ability to examine and define dependencies and links between model objects via a matrix
 - include the ability to edit object properties within diagram symbols
- Flexible diagram printing capabilities
 - visible page boundaries on canvas
 - print whole diagram
 - print specific pages
 - fit to page
- Find objects in local models and in a repository by searching names or other properties
- Replace selected text in object names and other properties
- Save diagram images as graphics files in a variety of standard formats
- Group related models and other files that you need for a particular project in a container that can be shared with others and managed as a simple entity in a repository
- Defined browser workspaces to hold working sets of models and projects
- Annotate diagrams with additional symbols to improve communication
- Expert, timely support available

Interfaces and Integration

- Import existing data models created by other tools
- Reverse engineer existing data artifacts, such as XML schemas and database schemas, into physical data models
 - includes the ability to reverse engineer database schemas from data definition language (DDL) files
- Generate or update an external data artifact, such as an XML or database schema, from a data model
 - the combination of this feature and the previous feature is called ‘round-trip engineering’, and is *the* key capability when managing external data artifacts
 - includes the ability to generate data definition language (DDL) files
- Create or update a model based upon information held in spreadsheets
- Compare two data models of the same or different types, and update one or both models as a result (this may also be referred to as merging models)
 - include comparing two versions of a model
 - preview, print, and save comparison results
- Compare a data model with an existing data artifact, such as an XML or database schema, and update the model and/or the data artifact as a result
- Export data models in a format that can be imported into other tools
- Generate test data to populate a test database or XML document
- Build cross-references or traceability links between data model objects and objects defined in other types of models, such as business process or enterprise architecture models
- Build cross-references or traceability links between data model objects and Business Requirement models
- Build cross-references or traceability links between data model objects and shared Business Rules
- Integration with popular development environments
- Integration with process and portfolio management workflows, and with IT configuration management
- Generate scripts to manage the movement of data through the archiving cycle, or data movements

Tool Management and Communication

- Extend the tool’s underlying data model, allowing analysts to change the way in which model objects are defined and to define new types of model objects
- Extract information from model objects for publication in various formats, such as html and documents
- Provide access to the content of models via a programmable interface, to provide a mechanism for the automation of repetitive tasks, and to extend the functionality provided by the tool
- Integrate with LDAP/Active Directory for user authentication

Collaboration

- A shared location for the storage of and controlled access to data models
- Control access to models by multiple analysts
- Control access to subsets of a model, perhaps individual model objects
- Provide direct access to models and links between models via a portal designed for use by non-specialist users of data models
- Allow selected non-modelers to review or revise definitions via a portal
- Resolve potentially conflicting changes made by different analysts
- Share and apply a 'house style' for the appearance and content of data models
- Share reference models and common business rules via a repository
- Allow anyone to subscribe to a model to know when models are checked-in

EXERCISE 7: Examining Your Current Data Modeling Tool

Examine the list of tool features above, remove any that don't matter to you, and assess the priority of each of the remaining features. Now add more detail to the feature list, and convert it into a real list of tool requirements. Re-examine the features you decided weren't relevant for you; I'm sure you'll re-instate some of them.

Does your current toolset support your new requirements list? Read the rest of the book, and discover the key PowerDesigner features that provide support for your needs.

Key Points

- Whiteboards, sticky notes, and spreadsheets are great for capturing information during a workshop, but you need a specialized data modeling tool to capture and use data models effectively.
 - The key features of a data modeling tool can be categorized as follows:
 - Core Modeling
 - Usability
 - Interfaces and Integration
 - Tool Management and Communication
 - Collaboration.
 - Build your own list of features, and check them against PowerDesigner's feature list on the Sybase web site² or in Chapter 9.
-

² See the Quick Reference Guide for the link